

Workflow Algorithms

Active Unified Code

1 Scope

This document describes the active workflow algorithms for:

- `UnifiedFramework/DATA3/ExperimentalDataAnalysis/UnifiedCode/unified_codebase_runfile.py`
- `UnifiedFramework/DATA3/ExperimentalDataAnalysis/UnifiedCode/unified_codebase_library.py`
- `UnifiedFramework/DATA3/ExperimentalDataAnalysis/UnifiedCode/conductivity_paper.py`

Archived compatibility wrappers are intentionally excluded from the runtime algorithm.

2 Top-Level Runtime Algorithm (`unified_codebase_runfile.py`)

Inputs

- CLI flags (`-profile`, `-file-path`, `-selector`, `-run-doe`, `-calc-cov`, `-plot`, `-convert-to-concentration`, `-nfe`, `-solver`).

Algorithm

1. Parse CLI arguments.
2. Resolve `run\_doe` with profile defaults:
 - `DATA1 -> False`
 - `DATA2 -> False`
 - `DATA3 -> True`
 - Explicit CLI value overrides defaults.
3. Branch by profile:
 - `DATA1`: iterate over preset `DATA1 .mat` files (or user override), build `UnifiedPipelineConfigV24` per file.
 - `DATA2`: iterate over preset `DATA2 .mat` files (or user override), build `UnifiedPipelineConfigV24` per file.
 - `DATA3`: build one XLSX-oriented `UnifiedPipelineConfigV24` from preset/defaults or CLI overrides.
4. For each built config, call `run\_unified\_pipeline\_v24(config)`.

5. Print run summary:
 - load/validation state,
 - parameter labels and counts,
 - optional ParmEst results,
 - optional DoE/FIM results.

Output

- Console output only; all substantive results come from the `run\_unified\_pipeline\_v24` return dictionary.

3 Unified Pipeline Algorithm (`run\_unified\_pipeline\_v24` in `unified_codeb`)

Inputs

- `UnifiedPipelineConfigV24` (file path, selector, load options, model options, stage toggles, solver controls, optional report hook).

Algorithm

1. Load experiment through file-agnostic loader `load\_experiment\_easy(...)`:
 - detect source type (`.xlsx/.xls` vs `.mat`),
 - load source-specific structure,
 - validate loaded structure,
 - optionally apply spec overrides,
 - optionally convert conductivity signals to concentrations (XLSX path),
 - optionally quick-plot.
2. Resolve model options:
 - use explicit `config.model\_options` if provided,
 - otherwise use source-aware defaults (MAT and XLSX defaults differ).
3. Build experiment wrapper object: `DiafiltrationExperimentV24(exp, options)`.
4. Build labeled Pyomo model via `get\_labeled\_model()`.
5. Build metadata record (`\_build\_unified\_run\_metadata\_v24`).
6. Initialize base output dictionary with:
 - loaded experiment,
 - validation info,
 - resolved options,
 - model/object handles,
 - theta names,

- label counts,
 - run metadata.
7. If `config.run\_\_parmes` is true:
 - `run\_run\_unified\_estimation\_stage\_v24 -> estimate\_parameters\_with\_\_parmes\_v24(...)`
 8. If `config.run\_\_doe` is true:
 - `run\_run\_unified\_doe\_stage\_v24 -> run\_doe\_with\_pyomo\_v24(...)`
 9. If `config.report\_hook` exists, call it with the final output dictionary.
 10. Return the output dictionary.

Output

- A structured dictionary containing load, model, estimation, and DoE artifacts.

4 File-Agnostic Loading Algorithm (`load\_experiment + load\_experiment\_`

4.1 `load\_experiment(path, selector, ...)`

1. Convert `path` to a Path object.
2. Detect source type:
 - XLSX if `.xlsx/.xls`
 - MAT if `.mat`
 - otherwise fail.
3. Route to source-specific loader:
 - XLSX -> `load\_from\_xlsx(file\_path, sheet\_selector=selector, ...)`
 - MAT -> `load\_from\_mat(file\_path, struct\_key=selector)`
4. Run `validate\_experiment(exp)`.
5. Return `(exp, validation)`.

4.2 `load\_experiment\_easy(...)`

1. If selector absent, apply convenience defaults:
 - XLSX -> first sheet (0)
 - MAT -> "data_stru"
2. Call `load\_experiment(...)`.
3. Apply optional field overrides (`specs`) directly to the `ExperimentalData` object.
4. Optional conductivity-to-concentration conversion:

- only for XLSX,
 - append informational warning if MAT conversion is requested.
5. Optional quick plot.
 6. Return (`exp`, `validation`).

5 XLSX Loader Algorithm (`load\_from\_xlsx`)

1. Read workbook sheet via selector.
2. Parse metadata and tables:
 - time-series table,
 - vial data table,
 - calibration block,
 - optional assay/ICP table,
 - salt/component hints.
3. Canonicalize columns and data types.
4. Segment rows by vial swaps (`segment\_indices\_by\_swap`).
5. Build one `VialData` object per segment (`build\_vial\_from\_slice`).
6. Attach sheet-level metadata to `ExperimentalData`.
7. Fill defaults/provenance markers for missing optional metadata.
8. Apply initial-guess DB if provided and compatible.
9. Return fully assembled `ExperimentalData`.

6 MAT Loader Algorithm (`load\_from\_mat`)

1. Read `.mat` file (`read\_mat\_file`).
2. Select struct key (default key behavior if selector absent).
3. Recursively convert MATLAB structs to Python primitives (`mat\_struct\_to\_python`).
4. Map legacy MAT fields into canonical `ExperimentalData` + `VialData` schema.
5. Normalize arrays, scalars, units, and legacy naming conventions.
6. Return canonicalized `ExperimentalData`.

7 Conductivity-to-Concentration Algorithm (`apply\_conductivity\_to\_con`)

Inputs

- `ExperimentalData` object with conductivity signals in uS/cm.
- Conversion model choice (`auto`, `variant\_shedlovsky`, or `msa`).

Algorithm

1. Determine temperature (`temp\_K` override or `exp.Temp\_K`).
2. Choose model:
 - `auto`: infer cation count from salt names,
 - ≤ 2 cations $\rightarrow$ variant Shedlovsky,
 - ≥ 3 cations $\rightarrow$ MSA.
3. Pre-autofill model parameters:
 - Shedlovsky parameters inferred from salt identity when possible,
 - MSA vector parameters inferred for known salts when possible.
4. For each vial and each signal type (`retentate`, `permeate`):
 - verify units are `uS/cm`,
 - call `\_conductivity\_to\_concentration\_series(...)`,
 - write converted concentration array, units, and model tag back into vial fields.

7.1 `\_conductivity\_to\_concentration\_series(...)` detailed algorithm

1. Import `conductivity\_paper.py` helpers.
2. Convert conductivity units (`uS/cm` $\rightarrow$ `mS/cm`).
3. Build pointwise inverse map by root-finding on the forward conductivity model:
 - for Shedlovsky: invert `variant\_shedlovsky([conc\_M], ...)` over the concentration bracket,
 - for MSA: invert `msa\_transport(...)` over the total concentration bracket.
4. For each signal point:
 - if NaN, keep NaN,
 - else invert the forward function using `\_invert\_monotone\_1d` and write concentration,
 - if inversion fails, write NaN (best-effort conversion).
5. Convert output units if needed (`M` $\leftrightarrow$ `mM`).
6. Return concentration series.

8 Conductivity Physics Algorithm (`conductivity\_paper.py`)

8.1 `\_shedlovsky`

1. Compute constants (`B`, `q`, `B1`, `B2`).
2. Compute ionic strength per concentration point.
3. Compute equivalent conductivity via the Shedlovsky correction expression.
4. Return equivalent conductivity array.

8.2 `variant\_shedlovsky`

1. Call `\_shedlovsky` for equivalent conductivity.
2. Convert molar concentration to equivalent concentration.
3. Convert equivalent conductivity to specific conductivity.
4. Return specific conductivity in `mS/cm`.

8.3 `msa\_transport`

1. Construct species number densities for one, two, or three salts.
2. For each concentration point:
 - compute mobilities and transport numbers,
 - solve alpha roots (bisection),
 - compute hydrodynamic and relaxation corrections,
 - compute species conductivity and sum bulk conductivity.
3. Convert to `mS/cm` and return.

9 Modeling + Estimation + DoE Algorithm (Core Stage Path)

9.1 Model assembly

1. Build the dynamic model using source-aware v24 constructor wrappers.
2. Apply discretization (`nfe`, `scheme`).
3. Attach weighted objective and labeled suffixes for ParmEst and DoE.

9.2 Parameter estimation (`estimate\_parameters\_with\_parmest\_v24`)

1. Build ParmEst experiment list wrappers.
2. Run estimation with the configured solver and options.
3. Optionally compute covariance.
4. If the configured path fails, use internal fallback/restart helpers.
5. Return estimated parameters, objective, and optional covariance artifacts.

9.3 DOE (`run\_doe\_with\_pyomo\_v24`)

1. Build the DoE object from experiment/model labels.
2. Execute the finite-difference sensitivity/FIM workflow.
3. Compute the D-optimality metric.
4. Return the FIM and summary metrics.

10 Active Workflow Summary

1. `unified\_codebase\_runfile.py` builds profile-specific configuration.
2. `run\_unified\_pipeline\_v24` orchestrates load -> model -> optional estimate -> optional DoE.
3. `conductivity\_paper.py` is used indirectly through conversion wrappers for XLSX conductivity workflows.
4. Archived wrappers are not part of the active algorithmic execution path.